

# Volume Preservation on skinned articulations with Reinforcement learning

Mauro Lopez - XCS229ii - December 2020

**Abstract**— The technique called “skeletal subspace deformation”, also known as linear blend skinning (LBS, also called smooth skinning) is widely used by character deformations for its easier use and its low computational cost. However this method has problems deforming the skin near joints bend, due to volume loss.

It’s a common technique that an artist adds extra joints and “animate” them by hand to solve the volume loss and create visually appealing articulation deformation. But this is a time consuming process and needs to be re-done for each character

We show that well-known reinforcement learning (RL) methods can be adapted to learn robust control policies capable of, given a set of limb examples, could learn to move provided extra joints to compensate for the volume loss.

**Keywords**— Volume preservation, linear blend skin, reinforcement learning

## I. INTRODUCTION

In character animation, skinning is the process of defining how the geometric surface of the character deforms according to a function of the skeletal poses.

Creating believable and compelling skin deformations is one of the main challenges of animated feature films, computer games, and interactive applications.

Traditionally, bone transformations describe the position and orientation of the joints, and the skin deformation is computed by linearly blending bone transformations to the skin. This technique is called “skeletal subspace deformation”, also known as linear blend skinning (LBS, also called smooth skinning) However, such simple and linear blending to the bone transformations cannot be expected to capture complex deformations.

Moreover, because the deformation is restricted to the subspace of the affine transformation of the bones, this method has problems deforming the skin near joints due to volume loss like is shown in the Fig 1. The default skin generates that volume loss artifact, in order to be fixed that needs several hours of artistic input.

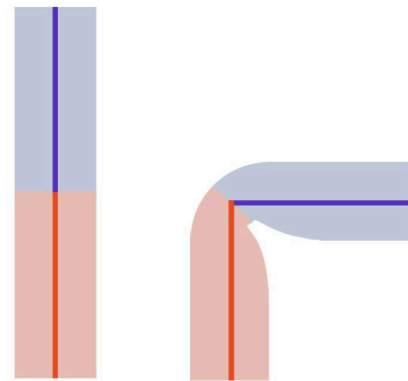


Fig. 1 On the left a simple limb, two segments with an articulation in the middle. On the right the volume lost generated by the linear smooth skin deformer

Physics based simulations manage to bring skeleton-based animation beyond by simulating secondary motions like jiggling, volume preservation and contact deformation effects. These secondary motions enrich the visual experience of the animation and are essential for creating appealing character animation. Despite offering such realistic effects, physically based simulation is computationally demanding and complex, thus it is usually avoided in interactive applications. The key challenge of producing believable deformations is to satisfy the conflicting requirements of real-time interactivity and achieving believable deformation, volume preservation and self-collision (skin contact) deformations.

Several approaches have been done attempting to solve this matter, those can be classified into two categories: learning based and direct deformation algorithms.

The direct deformation algorithms attempt to improve the skinning calculation itself, Position-based Skinning [4] and the Implicit Skinning [3] generate internal geometry to compute the volume loss, bulges and wrinkles. In both systems the mesh is first deformed using a standard linear skinning, then is corrected through the projection of its vertices to the internal volume mesh. This direct deformation approach is suitable for film or animation since the final result is not expected to be realtime. But this type of algorithm doesn't seem to be fast enough for real time environments like video games.

The learning based algorithms attempt to solve from rigs that already have the volume preservation fixed by any type of technique (artistic or physic base), then create a model that replicates the deformation but in real time. Skin from example [1] and Fast Deep Deformation Approximation [2] learn from fully rigged character, and it decomposes the deformation in two steps, a linear transformation (linear smooth skin) and a non-linear transformation. Solving the nonlinear part with a supervised learned model. By contrast Smooth Skinning Decomposition with Rigid Bones [5] doesn't expect a rig but just a  $N$  amount of examples of mesh deformation making it more flexible in how the volume preservation can be achieved. But the result is not a rigged character but a joint cloud, that can be suitable for video games since it reaches real-time deformation, but nearly impossible to edit afterwards.

In any case all learned based algorithms expect that the volume preservation is already solved, and the learned model is expected to be an optimization of the deformation.

None of the methods achieve both, solving the volume preservation and self collision with the minimal user input, and real time deformation.

The central contribution of our paper is a framework that, having already created extra joints for volume preservation, combines it with data

generated from the animation of bending limbs. It is possible to create a reinforced learned agent-joint that is capable of moving, preserving the volume and achieving simple self collision.

## II. METHODS

Our tasks will be structured as standard reinforcement learning problems, where an agent interacts with an environment according to a policy in order to maximize a reward. A policy  $\pi(a|s)$  models the conditional distribution over action  $a \in A$  given a state  $s \in S$ . At each control timestep, the agent observes the current state  $s_t$  and samples an action  $a$  from  $\pi$ . The environment then responds with a new state  $s' = s_t + 1$ , sampled from the dynamics  $p(s' | s, a)$ , and a scalar reward  $r_t$  that reflects the desirability of the action taken. For a parametric policy  $\pi_\theta(a|s)$ , the goal of the agent is to learn the optimal parameters  $\theta^*$  that maximizes its expected return

The policy gradient can therefore be interpreted as increasing the likelihood of actions that lead to higher than expected returns, while decreasing the likelihood of actions that lead to lower than expected returns. Our policies will be trained using the proximal policy optimization algorithm [5], which has demonstrated state-of-the-art results on a number of challenging control problems.

Building a joint-agent with our system involves three major steps. We begin with a limb rigged in an animation package such as Maya, then add the agent on the mesh place where it loses volume (this can be done by an artist or by an unsupervised algorithm such as PCA since the skincluster deformation is linear to the joints animation) and finally animate the limb bending (this animation could be random made or an artist could animate the bending to maximize the sample efficiency)

Given a reference animation, the goal of the policy is to move the agent in order to preserve the original binding distance of the agent to the closer

limb segment and avoid the intersection when the angle is bigger than 90 degrees

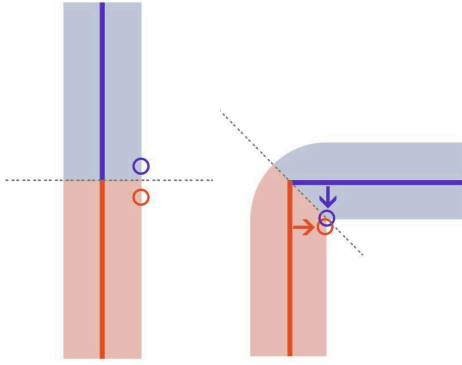


Fig. 2 On the left agents added on the problematic area. On the right the th agents learn to move away to the closer segment in order to preserve the inner volume

The state  $s$  describes the configuration of the limb's body, with features consisting of the relative positions of each agent with respect to the limb joints, the limb joint rotations expressed in quaternions. All features are computed in the local coordinate frame w.r.t each agent closest joint, and the agent z-axis facing towards the limb. Since the agent bind distance may vary, in different limbs, the bind vector is also included among the state features. The action  $a$  from the policy specifies the z-axis translation for each agent, describing the movement towards-away the closer limb segment.

Each policy  $\pi$  is represented by a neural network that maps a given state  $s$  and goal  $d$  to a distribution over action  $\pi(a|s,d)$ . The inputs are processed by two fully-connected layers with 1024, and 512 units each, followed by a linear output layer. ReLU activations are used for all hidden units. The value function is modeled by a similar network, with exception of the output layer, which consists of a single linear unit.

The reward  $r_t$  at each step  $t$  consists of two terms that encourage the asset to keep the binding distance to the closer segment, while also satisfying not invade the area of the other limb's segment like on the Fig 3

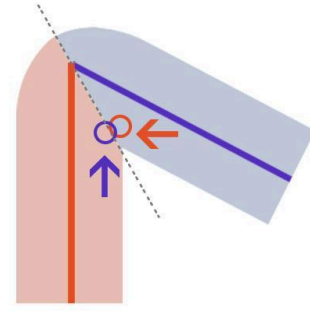


Fig. 3 As a simple self collision detection, if the blue agent invades the red area, it gets a negative reward, forcing the agent to move towards the blue segment, avoiding the intersection. And the same for the red agent

Our policies are trained with PPO using the clipped surrogate objective. We maintain two networks, one for the policy and another for the value function. Training proceeds episodically,

For each episode from the first frame until the last, it generates data by sampling actions from the policy at every step. This process is repeated as specified on the hyper parameter `num_episodes`. Once a batch of data has been collected, random sorted mini batches are sampled from the dataset and used to update the policy and value function. The value function is updated using target values computed with TD. The policy is updated using gradients computed from the surrogate objective, with advantages  $a_t$  computed using GAE (advantage value). Please refer to the PPO [5] reference material for a more detailed summary of the learning algorithm.

Since the agent needs to be more aware of the current action reward than the future, and because the data is randomly sorted, an aggressive discounted factor is added in order to make the agent short-sighted.

Once the agent learns the train process is being done again, updating the weights of previously learned policy models. This is done by the amount of times defined on the hyperparameter `num_trains`.

### III. RESULTS

A simple limb was evaluated with the proposed framework. The linear skin deformed was applied as default (no artistic intervention). Three agents were hand positioned where the limb loose volume. A simple animation was hand-made bending the limb  $140^\circ$  degrees, this animation-episode was 32 frames long. The episode was repeated 16 times for data collection, where each agent took actions, with a random gate value in order to get the reward. This generated a batch of 460 train samples and 52 validation samples for the episode.

Each episode batch is processed through the network updating the network policy and values. This process was repeated for 50 train sessions, the reward result plot is shown in the fig 4

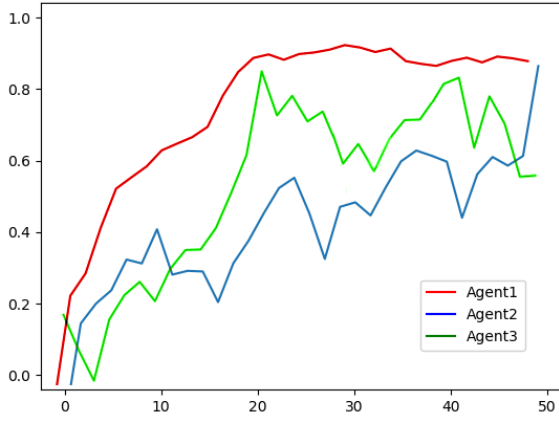


Fig. 4 The three agent reward for each training session, showing that all three agents learn how to preserve the volume and avoid self-collision maximizing the reward.

Visually the results are appealing, and the volume is preserved shown in the Fig 5

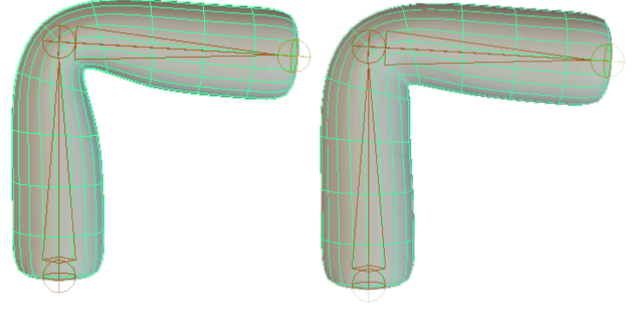


Fig. 5 On the left the limb bedding without the agent's influence. At the right the agents influencing the deformation, preserving the volume on being interior

But in order to measure the visual impact, a hand made animation for the agent was done by an artist to compare the model performance vs artistic, the distance between them is plotted in the Fig 6

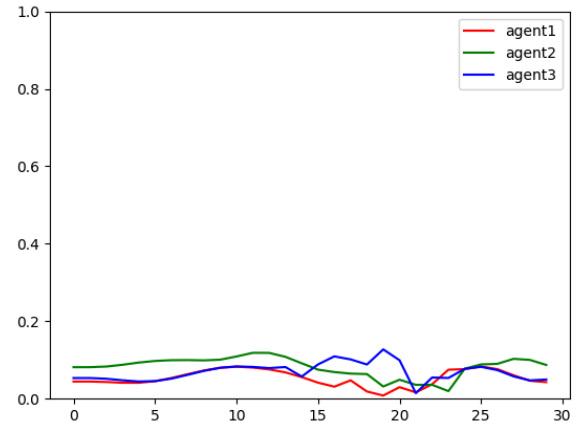


Fig. 6 The three distance to the hand made animation by each frame, showing that the agents on general lie are very close (not more than .1 deviation) to the hand made joints on most of the frames

#### IV. DISCUSSION

This is the first proof of concept of an unsupervised method that generates agents to solve volume preservation and simple self collision, without artistic intervention. Our framework proves that an agent can learn to preserve the volume without adding deformation complexity. This way realistic deformations in real-time can be achieved. Right now the only process needs artistic intervention is the vertex skin weights to the agents, that could be automatized with the same least square process used on Position Based skinning [4]. Furthermore, this is an initial setup, and could be expanded to achieve more complex behaviors, such as muscle bulging or skin sliding. Because in this framework the volume preservation is a naive implementation that could and needs to be improved. As is for now the volume is calculated with the agent's binding distance to the closer segment, and that distance will vary between different limbs. Also complex actions can be achieved like surface expansion at collision or set multi-agents that communicate the volume loss to the closest agent in order to expand/contract preserving the global limb volume.

#### REFERENCES

- [1] Building Efficient, Accurate Character Skins from Examples: [Alex Mohr, Michael Gleicher] - ACM Transactions on Graphics Vol 22 No 3 - Publication date: June 2003
- [2] Fast and Deep Deformation Approximations : [Stephen W. Bailey, Dave Otte, Paul D'Iorio, James F. O'Brien] - ACM Transactions on Graphics Vol. 37, No. 4, Article 119. Publication date: August 2018.
- [3] Implicit Skinning: Real-Time Skin Deformation with Contact Modeling [Rodolphe Vaillant, Loic Barthe, , Gael Guennebaud, Marie-Paule Cani, Damien Rohmer, Brian Wyvill, Olivier Gourmel, Mathias Paulin] - ACM Transactions on Graphics Vol. 32, No. 4 Article No.: 125 Publication date: July 2013
- [4] Position-Based Skinning for Soft Articulated Characters: [Nadine Abu Rummanm, Marco Fratarcangeli] - COMPUTER GRAPHICS forum Volume 34 (2015), Number 1 pp. 1–11 - Publication date: February 2015
- [5] Smooth Skinning Decomposition with Rigid Bones: [Binh Huy Le, Zhigang Deng] - ACM Transactions on Graphics Vol. 31, No. 6 Article No.: 199 Publication date: November 2012
- [6] Proximal Policy Optimization Algorithms: [John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov] - arXiv:1707.06347 Publication date: July 2017